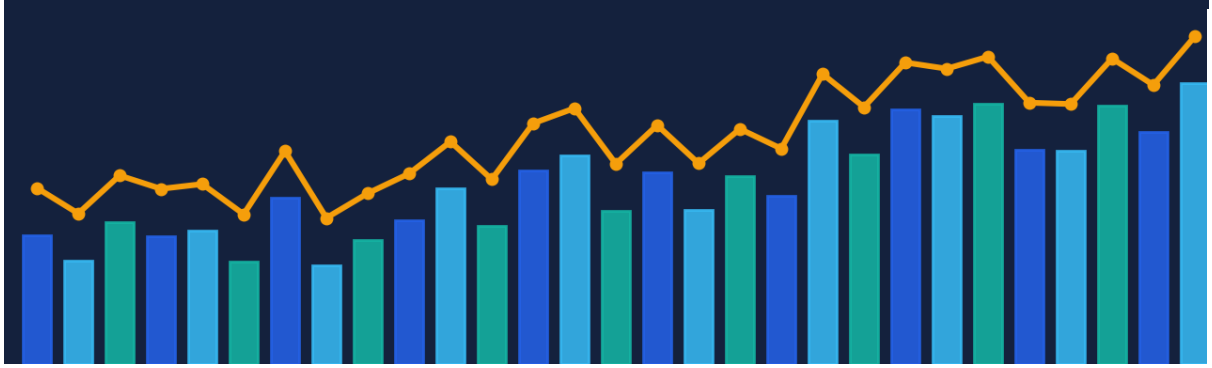
DATA ANALYTICS | PROJECT REPORT

Governed AI Data Analyst

From retail data to business insights using Python, MySQL, Power BI, Excel and a local AI model



Detail	Description
Prepared by	Krunal Panchal
Project guide by	Prof. Chirag Bhat
Qualification	B.Tech in Information Technology
Dataset	Sample Superstore, 9,994 rows and 21 columns
Technologies	Python, Pandas, MySQL, SQL, Power BI, DAX, Excel, Ollama, Llama 3.2:3b
Date	31 October 2026

Project at a glance

2.30M	286.4K	12.47%	793
Total sales	Total profit	Profit margin	Customers

5,009	458.61	299	9,994
Orders	Avg order value	Loss-making products	Rows analysed

Key finding

Furniture sells almost as much as Technology but earns only 2.49% margin, against about 17% for Technology and Office Supplies. The biggest profit opportunity in this dataset is fixing Furniture, not selling more of it.

Honest finding about the AI

The local model gave a wrong answer to the simple question “which category has the highest profit?”. Of 7 test questions, only 1 was fully correct. The report explains why and how to fix it (Section 09).

Contents

No.	Section	What it covers
01	About this project	Why I built it and the questions it answers
02	Dataset and tools	Data, technology stack and architecture
03	Cleaning the data	Python checks and first statistics
04	MySQL and SQL	Database, read-only user, category query
05	Power BI dashboard	Three pages and the DAX measures
06	Excel reports	Six-sheet business workbook
07	Local AI assistant	Ollama, Llama 3.2 and governed_ask()
08	Testing the assistant	Real answers checked against real numbers
09	Governance and audit log	Topic filter and request logging
10	Validation	Python, MySQL and Power BI compared
11	Problems, limits, next steps	What went wrong and what comes next
12	Conclusion	What I learned

01 About this project

Why I built it and which business questions it answers

I built this project to see how far I could take one dataset using the tools a data analyst actually works with. Most Superstore tutorials stop at a Power BI dashboard. I wanted to go further: clean the data in Python, store it in MySQL, query it with SQL, build dashboards, export Excel reports, and then try something newer by connecting a small language model that runs locally on my own laptop.

The AI part turned out to be the most educational, and not always because it worked. I tested it with real questions, compared its answers with my own numbers, and found several mistakes. I have kept those findings in this report because I think they say more about what I learned than a perfect demo would.

Questions the project answers

Category
Which product category makes the most profit?

Region
Which region sells the most, and which earns the most?

Losses
Which products are losing money?

Customers
Who are the best customers?

Discounts
What do discounts do to sales and profit?

Seasonality
How do sales change through the year?

What I built

Python analysis
Cleaning, KPIs and checks in Pandas

MySQL database
Structured storage and SQL queries

Power BI report
Three interactive dashboard pages

Local AI assistant
Llama 3.2 through Ollama, with filtering and logging

Note on units

Everything here uses the Sample Superstore dataset, which is practice data and not a live company. I kept its original currency and units and do not assume the amounts are rupees.

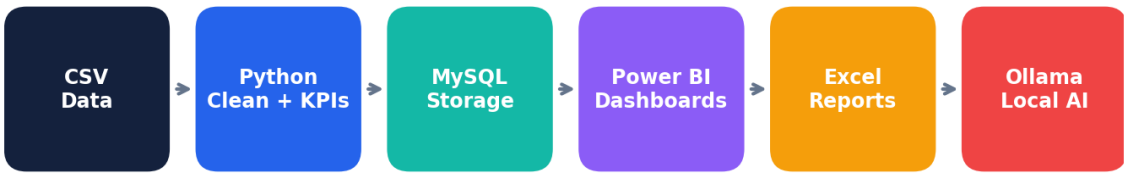
02 Dataset and tools

The data, the technology stack and how the pieces connect

The dataset has one row per order line. The columns I used most were Order ID, Order Date, Customer ID, Segment, Region, Category, Sub-Category, Product Name, Sales, Quantity, Discount and Profit.

Tool	What I used it for
Python + Pandas	Loading the CSV, quality checks, KPIs, connecting to MySQL and Ollama
MySQL + SQL	Storing the table (governed_ai_analytics.superstore) and running aggregation queries
Power BI + DAX	Three-page interactive dashboard and the KPI measures behind it
Excel	Multi-sheet workbook (Governed_AI_Analytics_Report.xlsx) for spreadsheet users
Ollama + Llama 3.2:3b	A small language model running locally to explain results in plain English
Jupyter Notebook	Where I wrote and tested all of the Python code

How the pieces connect



Python prepares the numbers and passes them to the AI. The AI never queries the database itself.

Figure 1: Project workflow

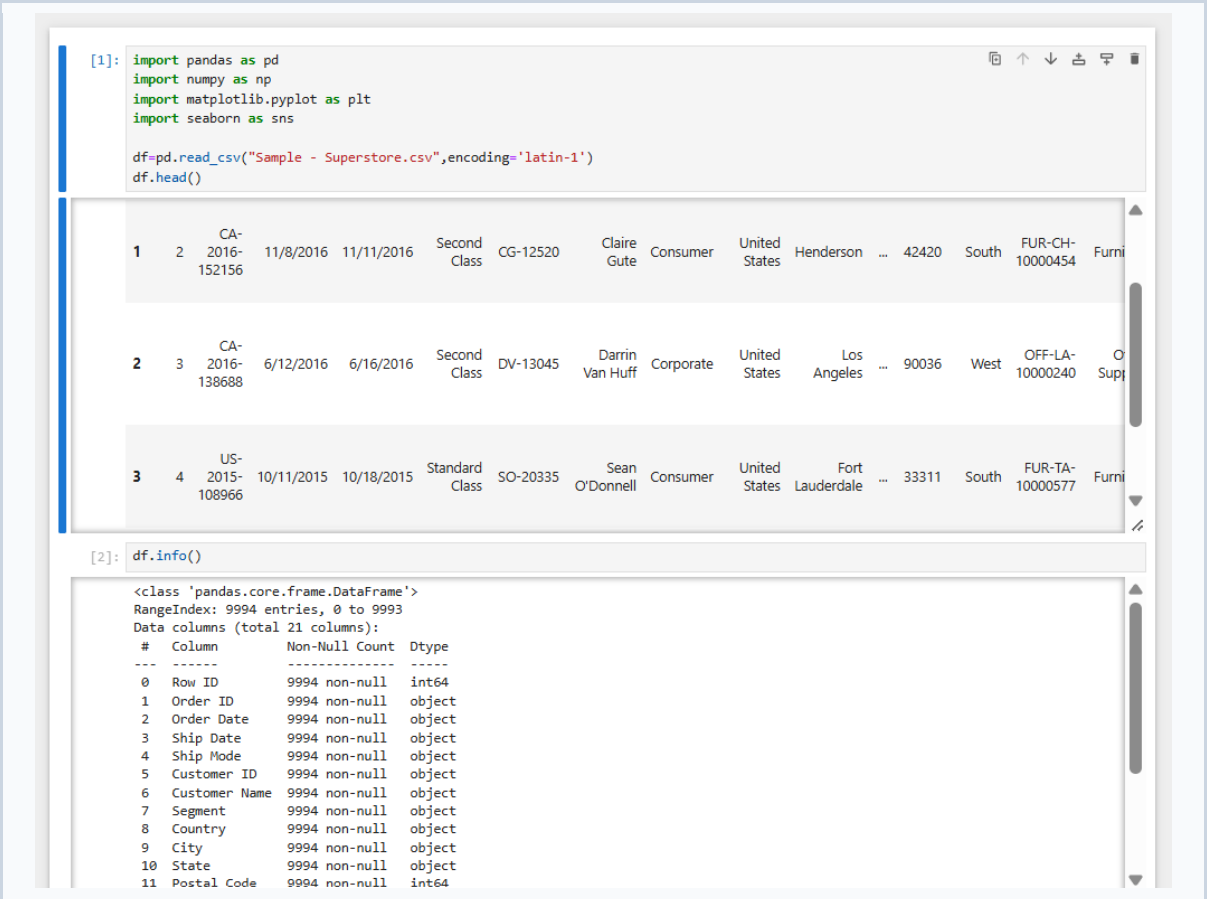
Important limit

The assistant does not write its own SQL or query the database live. It only sees a summary that my Python code prepares. This explains some of the results in Section 08.

03 Cleaning the data

Checking that the data can be trusted before analysis

Before any analysis I wanted to know whether I could trust the data. I loaded the file with Pandas (it needed encoding='latin-1' to open without errors) and inspected its structure.



```
[1]: import pandas as pd
     import numpy as np
     import matplotlib.pyplot as plt
     import seaborn as sns

     df=pd.read_csv("Sample - Superstore.csv",encoding='latin-1')
     df.head()
```

1	2	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	...	42420	South	FUR-CH-10000454	Furni
2	3	CA-2016-138688	6/12/2016	6/16/2016	Second Class	DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles	...	90036	West	OFF-LA-10000240	O Supp
3	4	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	...	33311	South	FUR-TA-10000577	Furni

```
[2]: df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9994 entries, 0 to 9993
Data columns (total 21 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Row ID         9994 non-null   int64
 1   Order ID       9994 non-null   object
 2   Order Date     9994 non-null   object
 3   Ship Date      9994 non-null   object
 4   Ship Mode      9994 non-null   object
 5   Customer ID    9994 non-null   object
 6   Customer Name  9994 non-null   object
 7   Segment        9994 non-null   object
 8   Country        9994 non-null   object
 9   City           9994 non-null   object
 10  State          9994 non-null   object
 11  Postal Code    9994 non-null   int64
```

Figure 2: Loading the dataset and running df.info()

The data has 9,994 rows and 21 columns, and every column I inspected shows 9,994 non-null values, so there are no gaps in the fields I looked at. I also wrote checks for duplicate rows, sales or quantity at or below zero, discounts outside 0 to 1, and orders shipped before they were placed.

```
[5]: df.shape
```

```
[5]: (9994, 21)
```

```
[6]: df.describe()
```

[6]:

	Row ID	Postal Code	Sales	Quantity	Discount	Profit
count	9994.000000	9994.000000	9994.000000	9994.000000	9994.000000	9994.000000
mean	4997.500000	55190.379428	229.858001	3.789574	0.156203	28.656896
std	2885.163629	32063.693350	623.245101	2.225110	0.206452	234.260108
min	1.000000	1040.000000	0.444000	1.000000	0.000000	-6599.978000
25%	2499.250000	23223.000000	17.280000	2.000000	0.000000	1.728750
50%	4997.500000	56430.500000	54.490000	3.000000	0.200000	8.666500
75%	7495.750000	90008.000000	209.940000	5.000000	0.200000	29.364000
max	9994.000000	99301.000000	22638.480000	14.000000	0.800000	8399.976000

```
[7]: df.columns.tolist()
```

```
[7]: ['Row ID',
      'Order ID',
      'Order Date',
      'Ship Date',
      'Ship Mode',
      'Customer ID',
      'Customer Name',
      'Segment',
      'Country',
      'City',
      'State',
      'Postal Code',
      'Region',
      'Product ID',
      'Category',
      'Sub-Category',
      'Product Name',
      'Sales',
      'Quantity',
      'Discount',
      'Profit']
```

Figure 3: Shape, descriptive statistics and column list

15.6%	54.49	-6,599.98	8,399.98
Average discount	Median sale	Largest loss (one line)	Largest gain (one line)

Those wide spread matters: a few large orders can move the totals a lot, so averages alone would mislead.

04 MySQL and SQL

Storing the data and querying it with SQL

I created a database called governed_ai_analytics with one table, superstore. I also created a separate user, ai_analyst, with SELECT permission only, so anything on the AI side can read data but never change it.

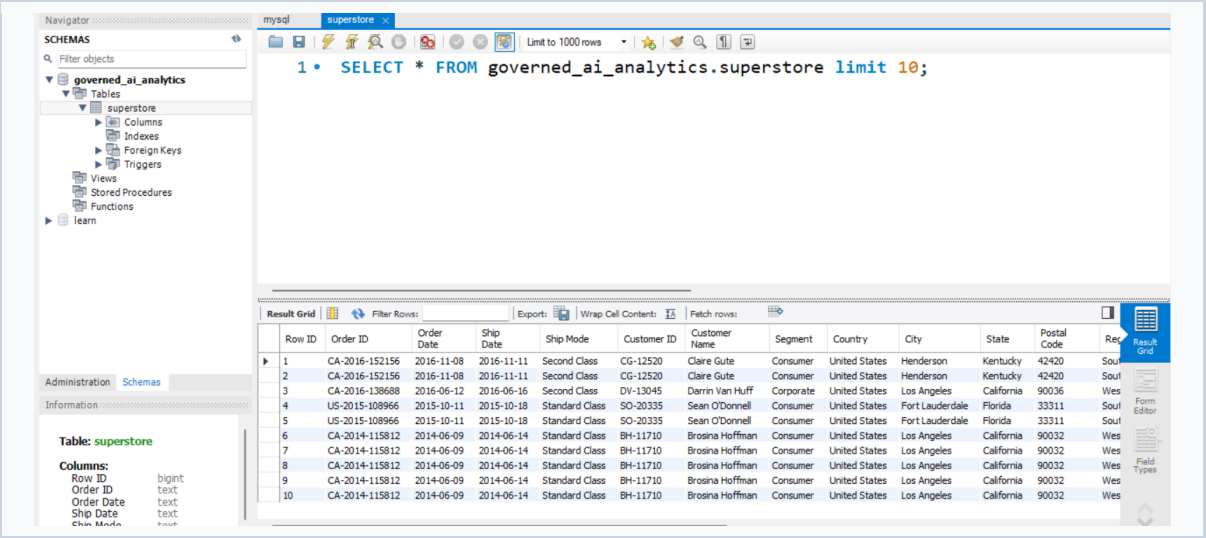


Figure 4: The superstore table in MySQL Workbench

One thing I noticed: Order Date and Ship Date were stored as text. It did not break anything, but for proper date work in SQL I would convert them to DATE columns.

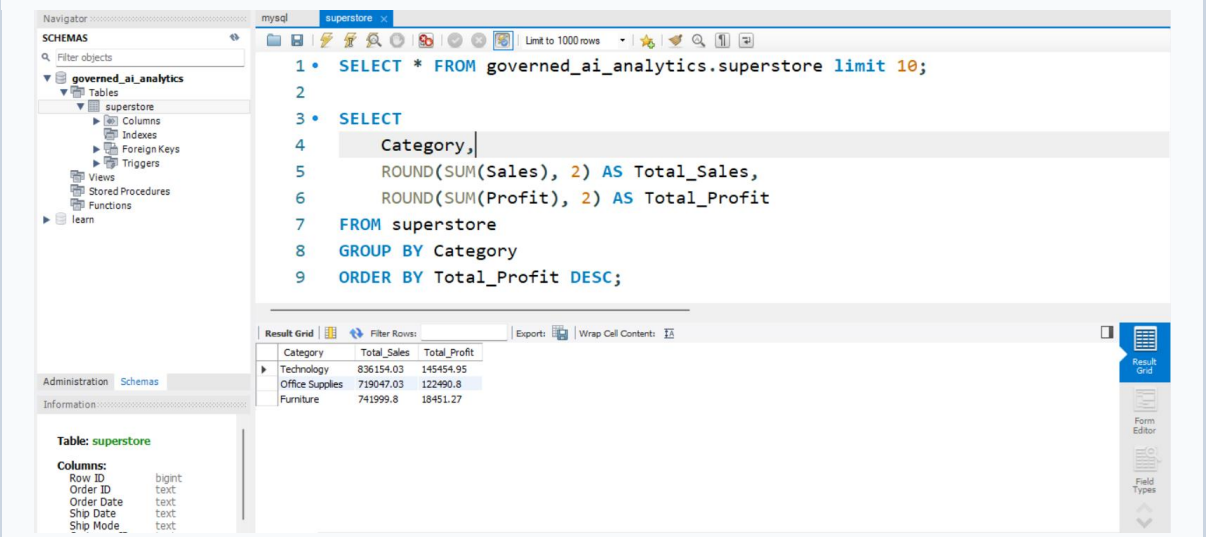


Figure 5: Category sales and profit with GROUP BY

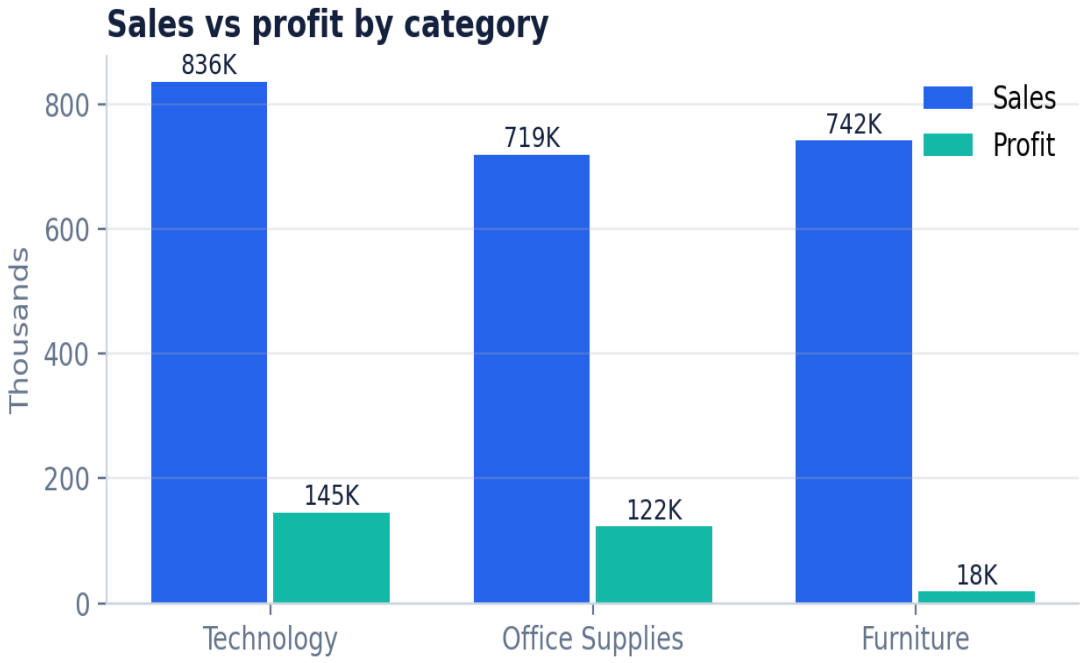


Figure 6: Sales vs profit by category (from the SQL result)

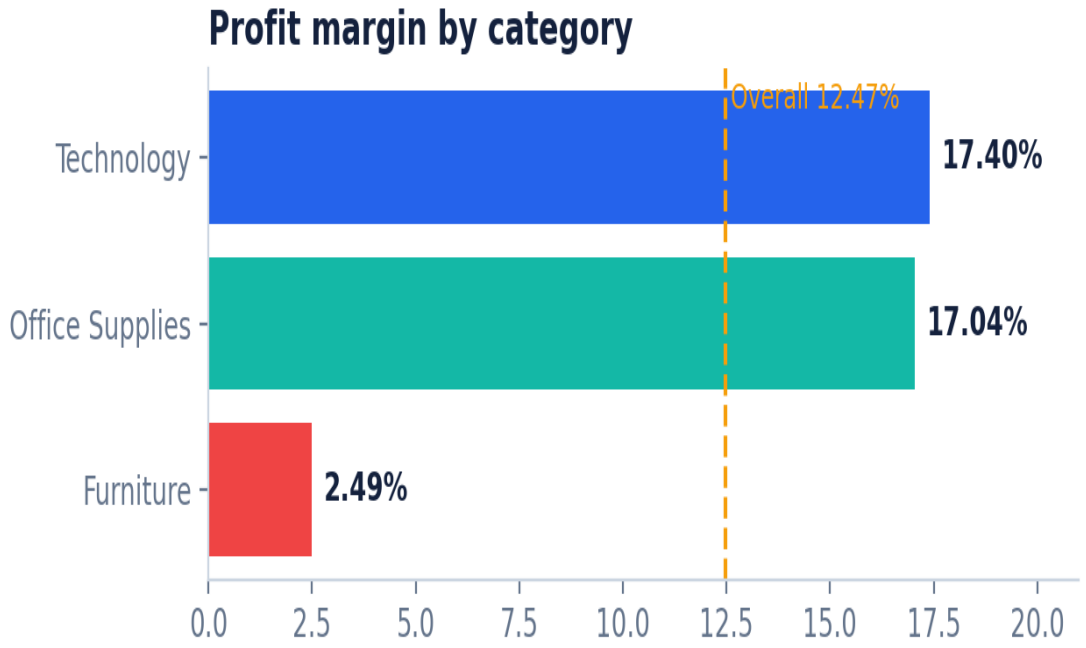


Figure 7: Profit margin by category

What the numbers say

Technology earns the most profit. Furniture sells more than Office Supplies but makes only 18.5K profit. The three rows add up to 2,297,200.86 in sales and 286,397.02 in profit, the same totals as Python and Power BI.

05 Power BI dashboard

Three interactive pages and the DAX behind them

The report has three pages with a consistent dark theme, KPI cards, and arrow buttons for navigation. Measures are written in DAX so every card and chart reacts to the slicers.

Measure	DAX formula
Total Sales	SUM(superstore[Sales])
Total Profit	SUM(superstore[Profit])
Total Orders	DISTINCTCOUNT(superstore[Order ID])
Total Customers	DISTINCTCOUNT(superstore[Customer ID])
Profit Margin %	DIVIDE([Total Profit], [Total Sales], 0) * 100
Average Order Value	DIVIDE([Total Sales], [Total Orders], 0)

Page 1: Executive Overview



Figure 8: Executive Overview (date slicer 03-01-2014 to 20-03-2017)

> **Why these cards look lower**
>
> The date slicer ends on 20-03-2017, so the cards show filtered values: 1.65M sales, 205K profit and 12.42% margin. The full-data figures (2.30M, 286.40K, 12.47%) are on the other pages. A slicer can change a headline number a lot.

Sales through the months of the year are low in the middle and climb sharply from September to December. Technology leads the category chart, and West has the highest regional sales.

Page 2: Profitability Analysis

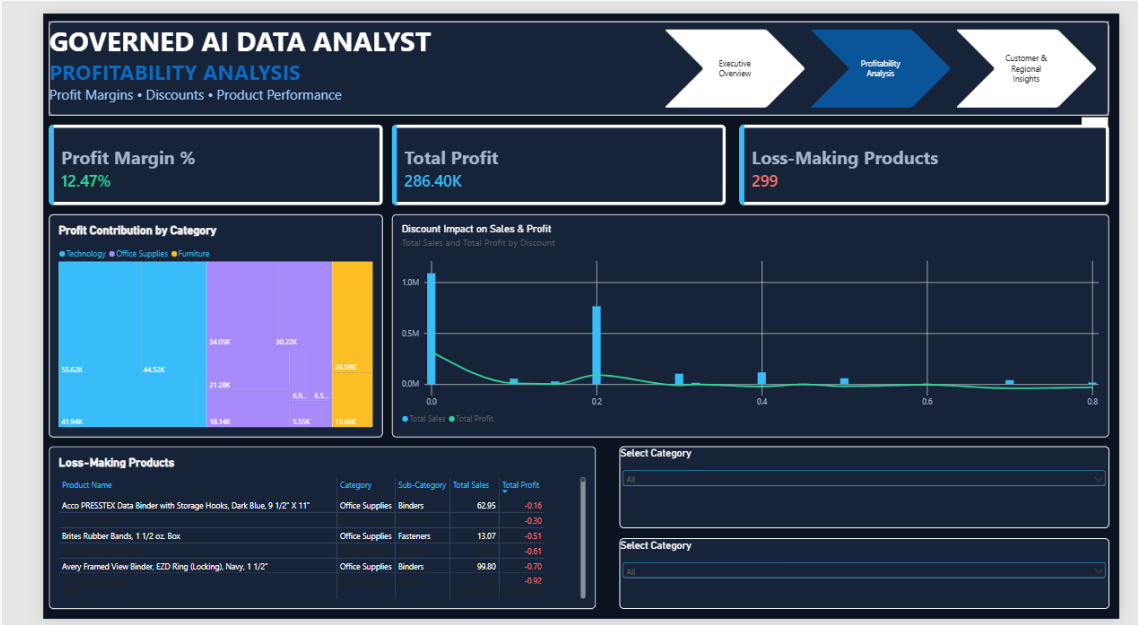


Figure 9: Profitability Analysis

This page answers where the business loses money. The margin is 12.47% and 299 products are loss-making. The discount chart shows most sales at 0% and 20% discount, with profit falling to zero or below as discounts rise. One chart does not prove cause and effect, but it suggests heavily discounted orders do not pay for themselves. The loss table lists cheap items such as binders and rubber bands, where a small discount can wipe out the margin.

Page 3: Customer and Regional Insights



Figure 10: Customer and Regional Insights

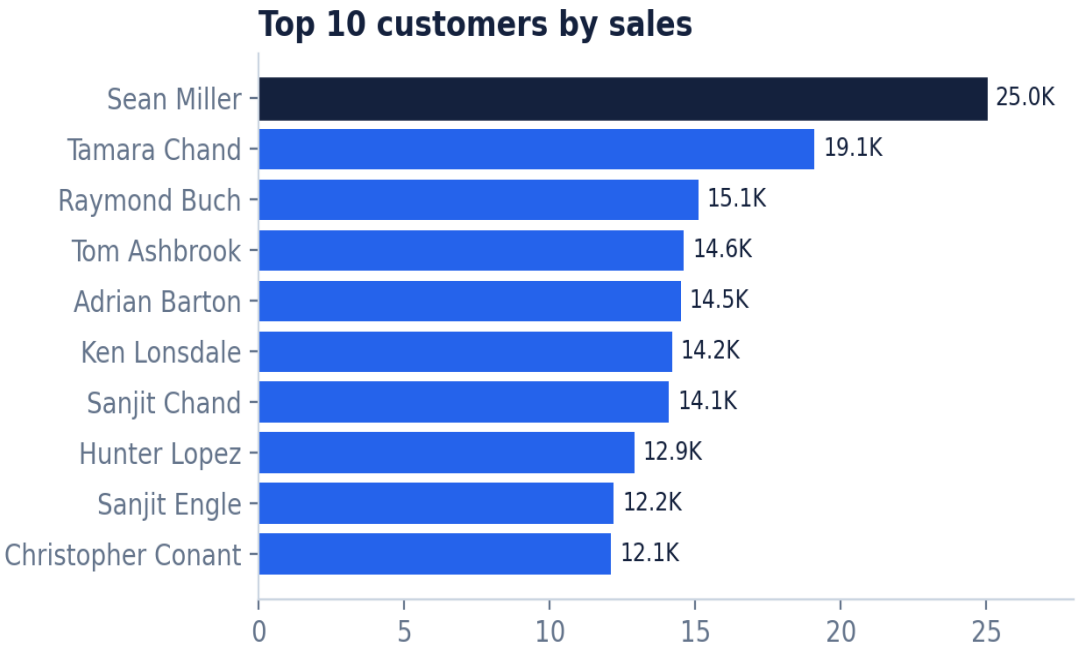


Figure 11: Top 10 customers by sales

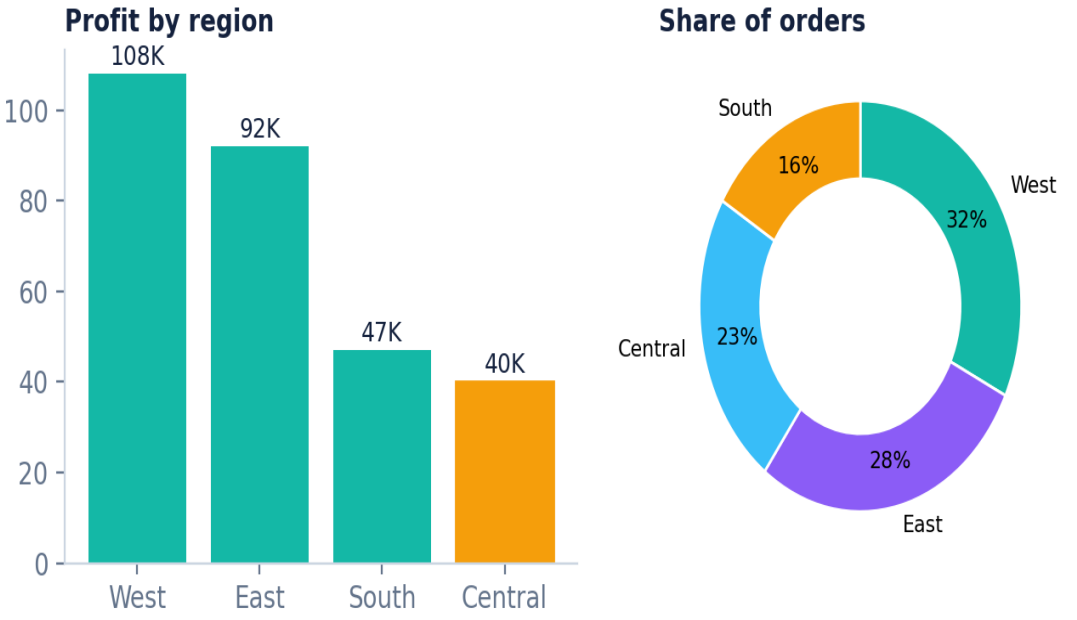


Figure 12: Profit by region and share of orders

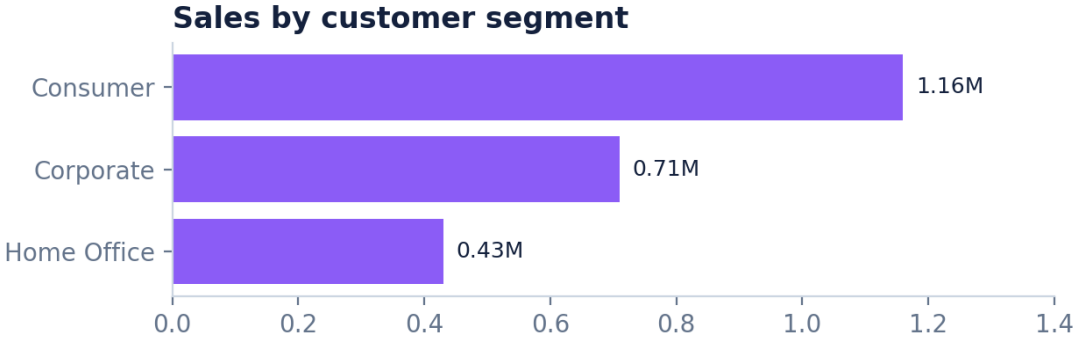


Figure 13: Sales by customer segment

- **Top customer:** Sean Miller at about 25.0K, well ahead of the next customers.
- **Segments:** Consumer brings 1.16M, ahead of Corporate (0.71M) and Home Office (0.43M).
- **Regions:** West leads on profit (108K) and orders (32%). Central has a bigger share of orders than South (23% against 16%) but earns less profit (40K against 47K), so it is worth investigating.

06 Excel reports

A six-sheet workbook for spreadsheet users

Some people would rather open a spreadsheet than a dashboard, so I exported a workbook called Governed_AI_Analytics_Report.xlsx with six sheets:

1 Category Performance
Sales and profit by category

2 Regional Performance
Sales, profit and orders by region

3 Top Customers
Highest-spending customers

4 Loss-Making Products
Products with negative total profit

5 Discount Analysis
Discount vs sales and profit

6 Monthly Sales Trends
Sales by month

Add screenshot

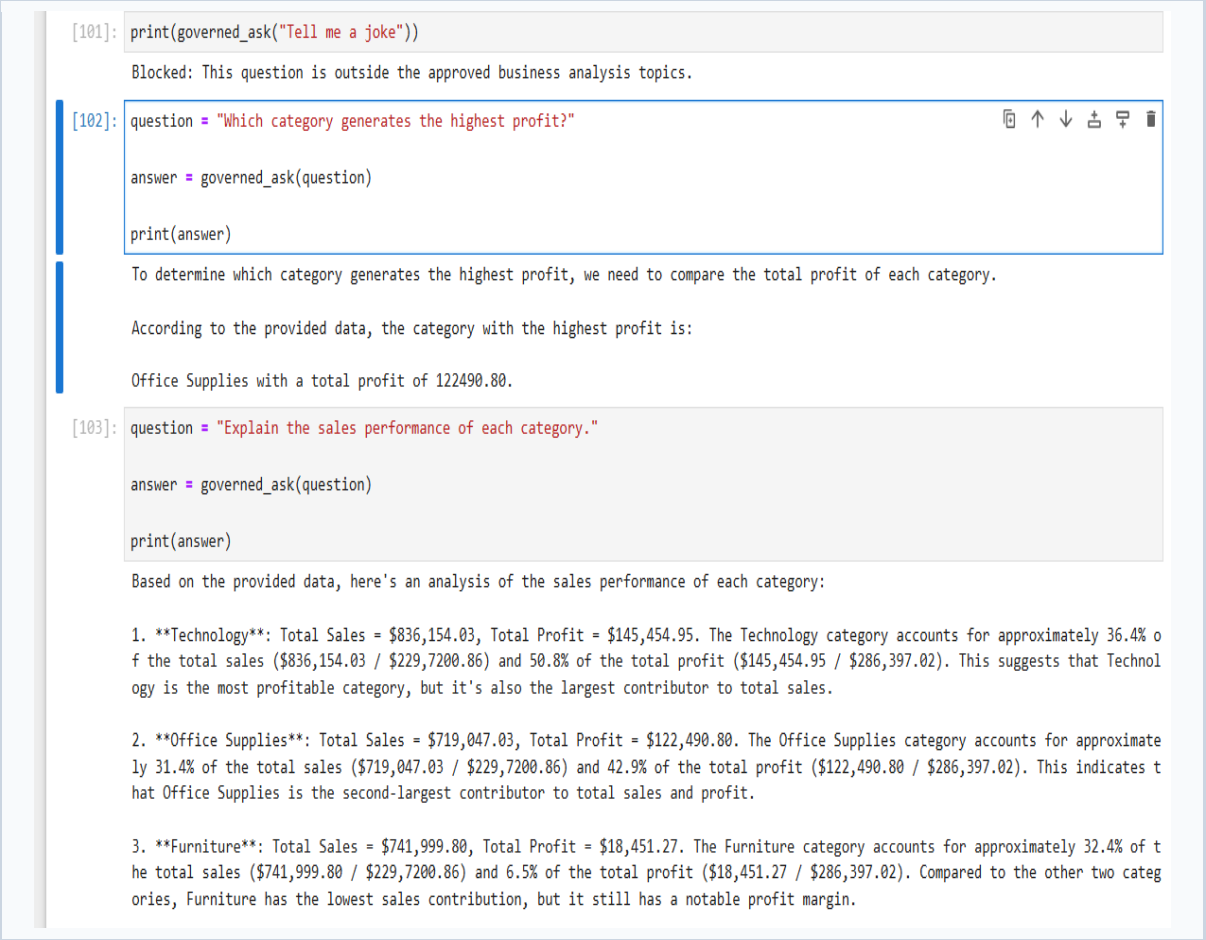
[Insert a screenshot of the Excel workbook with its sheet tabs here]

07 Local AI assistant

Running Llama 3.2 locally through Ollama

This is the part I was most curious about. Ollama lets me run a language model on my own machine, so no data leaves the laptop. I used Llama 3.2:3b, a small model that runs without a powerful GPU. The assistant works through a function I wrote called governed_ask():

Step	What happens
1	The question is checked against approved keywords: sales, profit, category, region, customer, product, order, discount.
2	If it is off topic, it is blocked and logged.
3	If allowed, Python sends the question to the model with the business summary and category table.
4	The model writes an answer, which is shown to the user and logged.



```
[101]: print(governed_ask("Tell me a joke"))

Blocked: This question is outside the approved business analysis topics.

[102]: question = "Which category generates the highest profit?"

answer = governed_ask(question)

print(answer)

To determine which category generates the highest profit, we need to compare the total profit of each category.

According to the provided data, the category with the highest profit is:

Office Supplies with a total profit of 122490.80.

[103]: question = "Explain the sales performance of each category."

answer = governed_ask(question)

print(answer)

Based on the provided data, here's an analysis of the sales performance of each category:

1. **Technology**: Total Sales = $836,154.03, Total Profit = $145,454.95. The Technology category accounts for approximately 36.4% o
f the total sales ($836,154.03 / $229,7200.86) and 50.8% of the total profit ($145,454.95 / $286,397.02). This suggests that Technol
ogy is the most profitable category, but it's also the largest contributor to total sales.

2. **Office Supplies**: Total Sales = $719,047.03, Total Profit = $122,490.80. The Office Supplies category accounts for approximate
ly 31.4% of the total sales ($719,047.03 / $229,7200.86) and 42.9% of the total profit ($122,490.80 / $286,397.02). This indicates t
hat Office Supplies is the second-largest contributor to total sales and profit.

3. **Furniture**: Total Sales = $741,999.80, Total Profit = $18,451.27. The Furniture category accounts for approximately 32.4% of t
he total sales ($741,999.80 / $229,7200.86) and 6.5% of the total profit ($18,451.27 / $286,397.02). Compared to the other two categ
ories, Furniture has the lowest sales contribution, but it still has a notable profit margin.
```

Figure 14: A blocked question, then the first allowed questions

08 Testing the assistant

Real answers checked against real numbers

I asked the assistant a set of questions and checked every answer against the MySQL and Python numbers. I am glad I did, because the results were mixed.

```
[104]: question = "Which category has the lowest profit margin?"

       answer = governed_ask(question)

       print(answer)

To determine which category has the lowest profit margin, we need to calculate the profit margin for each category.

Profit margin is calculated by dividing the total profit by the total sales and then multiplying by 100.

Let's calculate the profit margin for each category:

- Technology: (145454.95 / 836154.03) * 100 ≈ 17.44%
- Office Supplies: (122490.80 / 719047.03) * 100 ≈ 17.00%
- Furniture: (18451.27 / 741999.80) * 100 ≈ 2.49%

Based on the calculations, the Furniture category has the lowest profit margin, which is approximately 2.49%.

[105]: question = "What are the major business insights from this dataset?"

       answer = governed_ask(question)

       print(answer)

Blocked: This question is outside the approved business analysis topics.

[106]: question = "Which region performs better in terms of profit?"

       answer = governed_ask(question)

       print(answer)

To determine which region performs better in terms of profit, we need to compare the total profit of each category.

Based on the provided data, the Technology category has a total profit of 145454.95, while the Office Supplies category has a total
profit of 122490.80, and the Furniture category has a total profit of 18451.27.
```

Figure 15: Profit margin, a blocked question and a regional question

```
[108]: question = "What recommendations would you give to improve profitability?"

       answer = governed_ask(question)

       print(answer)

Based on the provided data, here are some recommendations to improve profitability:

1. **Focus on high-profit products**: Analyze the Category performance data, which shows that the total profit is significantly lowe
r in the Furniture category ($18451.27) compared to the Technology and Office Supplies categories. It may be beneficial to explore w
ays to increase profit margins in the Furniture category, such as renegotiating supplier contracts or developing new products that c
ommand higher prices.

2. **Improve profit margins in Office Supplies**: The Office Supplies category has a relatively high total profit of $122490.80, whi
ch is 53% of the total profit. To further increase profitability, consider optimizing inventory levels, renegotiating supplier contr
acts, or implementing cost-saving measures to reduce waste and inefficiencies.

3. **Monitor and optimize Technology category performance**: The Technology category accounts for the majority of total sales and to
tal profit. However, the total profit is still lower compared to the Office Supplies category. Review sales data and pricing strateg
ies to identify opportunities to increase profit margins in this category.

4. **Invest in marketing and customer acquisition**: The total number of customers is relatively low (793), which could be a limitin
g factor for growth. Consider investing in marketing and customer acquisition efforts to expand the customer base and increase avera
ge order value.

5. **Analyze and optimize operational costs**: The total profit is $286397.02, which is a relatively low margin. Review operational
costs, such as labor, logistics, and overhead expenses, to identify areas for cost reduction and optimize profitability.

By addressing these areas, the business can potentially increase profitability and improve overall performance.
```

Figure 16: Recommendations to improve profitability

```
[109]: question = "Which products are generating losses?"

       answer = governed_ask(question)

       print(answer)
```

```
Based on the provided data, to determine which products are generating losses, we need to compare the 'Total Profit' for each produc
t to the 'Total Sales' for that product.

Since 'Total Profit' is significantly lower than 'Total Sales' for most products, it indicates losses. However, the data does not pr
ovide the profit margins or the costs associated with each product, so we can only conclude that the products with higher 'Total Sal
es' and lower 'Total Profit' are likely generating losses.

From the category performance table, we can see that:

- Office Supplies have a relatively higher 'Total Profit' compared to 'Total Sales', indicating they are profitable.
- Furniture has a relatively lower 'Total Profit' compared to 'Total Sales', indicating they might be generating losses.

Please note that without additional data, we cannot determine the exact profit margins or the costs associated with each product, an
d therefore cannot definitively conclude that all losses are due to low profit margins or other factors.
```

```
[ ]:
```

Figure 17: The loss-making products question

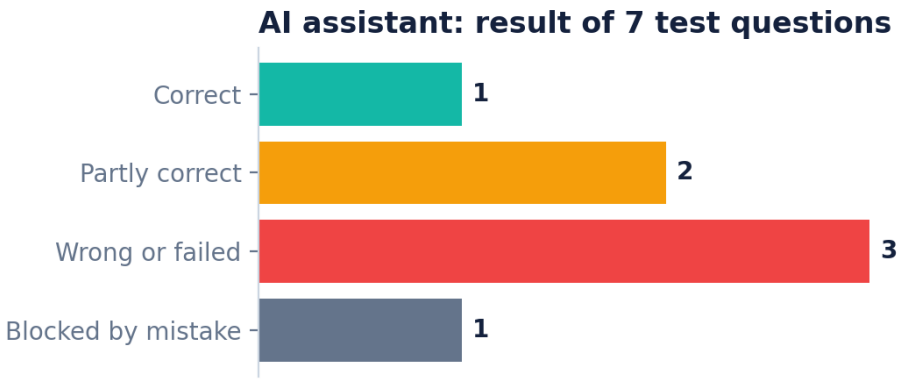


Figure 18: Scorecard of the 7 test questions

Question	Assistant said	What I found
Highest profit category?	Office Supplies, 122,490.80	**WRONG. Technology is highest at 145,454.95.**
Sales performance of each category	Correct figures and percentages	**PARTLY. It called Furniture lowest in sales; Office Supplies is lowest. Total was mistyped.**
Lowest profit margin?	Furniture, about 2.49%	**CORRECT. Other margins slightly off (17.44% and 17.00% vs 17.40% and 17.04%).**
Major business insights?	Blocked	**BLOCKED BY MISTAKE. A good question had no approved keyword.**
Which region is better on profit?	Answered with categories	**WRONG. No regional data was supplied.**
Recommendations to improve profit	Five general ideas	**PARTLY. Said Office Supplies is 53% of profit (about 43%) and that Technology profit is below it.**
Which products lose money?	Said it could not tell; hinted Furniture	**WRONG. Furniture is profitable; 299 loss-making products exist in Power BI.**

What I took from this

- **Small models are not reliable with numbers.** Most of the arithmetic was fine, yet it ranked categories wrongly in a simple question.
- **The model only knows what it is given.** Region and loss questions failed because I never passed that data in, and it guessed instead of saying it did not know.
- **A keyword filter is too blunt.** It blocked a good question and would let through an off-topic one containing a word like “order”.

> **Better design**
> Let Python calculate the answer (for example the top category) and use the model only to explain it in words. Then the numbers always come from code that can be verified.

09 Governance and audit log

Filtering questions and keeping a record

Even for a prototype, I wanted every question to leave a trace. The log_ai_request() function stores the timestamp, the question, ALLOWED or BLOCKED, and the reason.

```
[89]: import pandas as pd
from datetime import datetime

audit_log = []

def log_ai_request(question, status, reason):
    audit_log.append({
        "timestamp": datetime.now().isoformat(timespec="seconds"),
        "question": question,
        "status": status,
        "reason": reason
    })

    return "Request logged successfully."


log_ai_request(
    "Which category has the highest profit?",
    "ALLOWED",
    "Approved business analysis topic"
)

log_ai_request(
    "Tell me a joke.",
    "BLOCKED",
    "Outside approved business analysis scope"
)

audit_df = pd.DataFrame(audit_log)
display(audit_df)
```

Figure 19: The audit log function and sample entries

```
[91]: audit_df = pd.DataFrame(audit_log)

audit_df.to_csv(
    "Governed_AI_Audit_Log.csv",
    index=False
)

print("Audit log saved successfully.")
print("Total logged requests:", len(audit_df))
```

```
Audit log saved successfully.
Total logged requests: 4
```

```
[92]: import os
import pandas as pd

audit_file = "Governed_AI_Audit_Log.csv"

if os.path.exists(audit_file):
    saved_audit = pd.read_csv(audit_file)

    print("File verified:", audit_file)
    print("File size:", os.path.getsize(audit_file), "bytes")
    print("Saved records:", len(saved_audit))
    display(saved_audit.tail(5))
else:
    print("Audit log file not found.")
```

```
File verified: Governed_AI_Audit_Log.csv
File size: 408 bytes
Saved records: 4
```

	timestamp	question	status	reason
0	2026-10-31T18:41:20	Which category has the highest profit?	ALLOWED	Approved business analysis topic
1	2026-10-31T18:41:20	Tell me a joke.	BLOCKED	Outside approved business analysis scope
2	2026-10-31T18:41:20	Which category has the highest profit?	ALLOWED	Approved category-analysis topic
3	2026-10-31T18:41:54	Tell me a joke.	BLOCKED	Outside approved category-analysis scope

Figure 20: Audit log exported to CSV and verified (4 records)

The log does the basic job of showing who asked what and what happened. It is still a CSV file in a notebook folder, with no login, no roles and no protection from editing. A real system would need all of those.

10 Validation: do the tools agree?

Python, MySQL and Power BI compared

A big part of this project was making sure the tools agreed. I compared Python totals with Power BI (full date range, no other filters) and with the MySQL category query.

Metric	Python / MySQL	Power BI	Match
Total Sales	2,297,200.86	2.30M	**Yes**
Total Profit	286,397.02	286.40K	**Yes**
Total Orders	5,009	5K	**Yes**
Profit Margin	12.47%	12.47%	**Yes**
Total Customers	793	793	**Yes**
Average Order Value	458.61	458.61	**Yes**

6
Metrics compared

6 of 6
Matching

0
Mismatches

Power BI rounds its cards, so I compared at the displayed precision. Two values can be checked by hand: 286,397.02 ÷ 2,297,200.86 = 12.47%, and 2,297,200.86 ÷ 5,009 = about 458.61.

11 Problems, limits and next steps

What went wrong and where it goes from here

Problems I ran into

- **Reading the CSV.** The default encoding failed, so I used latin-1.
- **Dates in MySQL.** Imported as text; should be converted to DATE.
- **Slicers changing numbers.** My first dashboard page showed lower totals than the others. I now check slicer ranges first.
- **AI mistakes.** Covered in Section 08. Never trust a model's numbers without checking.
- **Topic filter.** It blocked a legitimate question.

Limitations

- Static sample dataset, not live company data.
- The assistant sees only a prepared summary and cannot run its own SQL or read Power BI.
- The keyword filter and CSV audit log are prototypes, not real security.
- No login or role-based access.
- AI answers must be verified before anyone relies on them.

What I would do next

Improvement	Idea
Function calling	Give the model approved Python functions (top category, regional profit, loss-making products) so numbers come from code.
Controlled Text-to-SQL	Turn questions into read-only SQL and validate every query before running it.
Better governance	Replace the keyword list, add user roles, store the log in a database.
Model test set	Write 20 to 30 questions with known answers and score the assistant.
Clean-up	Convert MySQL date columns and automate data refresh.

12 Conclusion

What this project taught me

This project took one retail dataset through the whole analytics process: checking the data, storing it, querying it, visualising it, reporting on it, and finally asking questions of it in plain English. The numbers matched across Python, MySQL and Power BI, which gave me confidence in the analysis.

The AI assistant was less reliable than the rest of the pipeline. It explained results in readable language but made factual errors that I only caught because I had the real numbers to check against. My main takeaway is that AI can help present analysis, but the figures should come from a source you can verify.

3	6	7	3
Dashboard pages	Excel sheets	AI test questions	Tools matched

Skills practised

Python and Pandas, data validation, MySQL and SQL, Power BI and DAX, Excel reporting, local LLM integration with Ollama, and basic AI governance.